

RESEARCH DIRECTIONS AND OPEN PROBLEMS AROUND CONDENSATION

SAM EISENSTAT

CONTENTS

1. Agents using condensation	1
1.1. Kolmogorov condensation	2
1.2. Online and out-of-distribution generalization	5
1.3. Value learning in toy settings	7
2. Existence of condensations	7
2.1. Almost perfect condensation	7
2.2. Other kinds of condensation	8
3. Condensation and statistical models	9
4. Language and logic	10
5. Further directions	10
References	10

1. AGENTS USING CONDENSATION

I'd like to analyze the ways that an agent might use the sorts of structures or representations belonging to the theory of condensation in order to organize their understanding. This broad topic is a major motivating application of condensation. That is, I think that certain ways that we would like to think about agents involve positing that they use certain representations, and condensation can help us with this, by guiding us as to what such representations we have reason to expect.

Before getting into further details, let's clarify a few things about how we will think about agency here. Generally, I'm going to use settings along the lines of those traditional in the analysis of agency in computer science and artificial intelligence. For example, we will have a sequence of observations at discrete time steps. Often I believe these settings have important limitations, but I will not discuss them here except to the extent that they are relevant to the exhibition of the principal phenomena to be studied.

Additionally, though we will aim at agency here, for the most part the practical complexity of the settings will be fairly minimal. We will look at agents doing things like predicting their observations, or settings like online learning, where we only care about immediate consequences, rather than coordinating activity over multiple time-steps, as would be the case in settings like reinforcement learning. Again, this is for simplicity and convenience—the intent is that the research directions discussed here should already be felt as posing real problems that matter within these simplifications.

1.1. Kolmogorov condensation. Before we can analyze an agent in this way, we need to match the structures of condensation to those we use to model agency. Here, we will model the observations of an agent as a sequence $(x_t)_{t \in T}$, where the idea is that x_t is the observation that the agent makes at time t . The set T of all times is a (finite or infinite) initial segment of the natural numbers. Each element x_t will be taken to be a finite string over the binary alphabet $\mathbf{2}$; as is conventional, we will thus write the space of all such strings as $\mathbf{2}^*$. The broader idea here is that these strings are “computational objects”—they are objects that the agent can act upon with certain operations, and by doing so pose certain hypotheses, which can be a guide to prediction and action.

This differs from the idea of a random variable model, as in Eisenstat (2025). To recall, a random variable model is a joint distribution over a set of random variables. A random variable model has the form $(\Omega, (X_i)_{i \in I})$, where Ω is a probability space, and $(X_i)_{i \in I}$ is a family of random variables on that probability space (satisfying some analytic conditions). In order to apply the ideas of Eisenstat (2025), we will thus need to replace this probabilistic condensation, defined in terms of (Shannon) entropy and mutual information, with an algorithmic-information, or Kolmogorov, condensation.

There is a direct analogy between entropy and algorithmic information. We want to replace the Shannon entropy $H(U)$ of a random variable U by the Kolmogorov complexity $K(u)$ of a string u . Correspondingly, we can replace relative entropies $H(U | V)$ and mutual informations $I(U; V | W)$ by algorithmic quantities $K(u | v)$ and $I(u; v | w)$. In information theory in general, this procedure often takes theorems to theorems, if applied with a little care. For example, we have

$$H(U | V) + H(V) = H(U, V) = H(V | U) + H(U)$$

and, as discussed by M. Li and Vitányi (2019), there are similar statements satisfied by algorithmic information. For “plain” algorithmic complexity, we have (Theorem 2.8.2 and Corollary 2.8.1—in the fourth edition at least)

$$C(u, v) = C(u) + C(v | u) + O(\log C(u, v)),$$

and using a certain variant Kc of conditional prefix complexity, we can do even better (Example 3.8.2):

$$Kc(u, v) = Kc(u) + Kc(v | u) + O(1).$$

We can list explicitly the ways that we have to take care here in passing to algorithmic information. Most obviously, we have to add the lower-order correction terms $O(\log C(u, v))$ and $O(1)$; the most straightforward analogue with no correction term is not necessarily true. Further, we should understand what is meant by these terms. While these terms bound the dependence of the error on u and on v , there is another parameter here—the choice of the *reference* universal Turing machine in the definition of algorithmic complexity. This is taken to be outside the quantifier of the asymptotic notation, so e.g. the $O(1)$ cannot grow without bound for a fixed reference machine, but it is not bounded with respect to also varying the machine. (If we want to, we might choose a reference machine to make this quantity small.) Finally, we should be careful about expressions with pairs, like $C(u, v)$. In the probabilistic setting, we would use (U, V) to refer to the product random variable. Here, $\langle u, v \rangle$ is a kind of pair data structure, and details like the

length of the string $\langle u, v \rangle$ depend on exactly what encoding convention we choose for such pairs.

Following the idea of this analogy, we'd like to give a definition of *latent string models* analogous to latent variable models, and we'd like an analogue of the objectivity theorem to hold. This should be something like the following. A string model is a family $(x_t)_{t \in T}$ as we discussed at the beginning of this section, and an associated latent string model is a pair of a family $(y_j)_{j \in J}$ and a contribution relation $\triangleright \subseteq J \times T$, where there's a distinguished element $\star \in J$, with $\star \triangleright t$ for all t , such that for all $t \in T$ we have $x_t = y_\star(y_j : j \triangleright t)$. In that last equation, we are interpreting the string $y_\star$ as a program—correspondingly the strings in $(y_j : j \triangleright t)$ are “input data”. Like with latent variable models, the idea is that each string is determined by the set of latents that contribute to it, but now we are replacing a probabilistic notion of determination by an algorithmic one. Now, following the proof of the objectivity theorem, we should get an algorithmic objectivity theorem for latent string models. Using the notation from the original objectivity theorem, this will say something like

$$Kc(y_{\supseteq A} \mid z_{\mathcal{F}^\star}) \leq \sum_{B \in \mathcal{F}} Kc(y_{\supseteq A} \mid x_B) + \sum_{v \in N} I(z_{\mathcal{L}(v)}; z_{\mathcal{R}(v)} \mid z_{\mathcal{I}(v)}) + \text{error terms},$$

or we could use C and other notions of mutual information if we prefer. (M. Li and Vitányi (2019) define

$$\begin{aligned} I_C(u : v) &= C(v) - C(v \mid u) \\ I(u : v) &= K(v) - K(v \mid u) \\ I(u; v) &= Kc(v) - Kc(v \mid u). \end{aligned}$$

This sort of thing would be satisfying, though it would also be good to better understand exactly what the error terms are, and how to keep them reasonably under control. It's possible that this would require some kind of coding tricks.

1.1.1. The contribution relation in Kolmogorov condensation. There's at least one thing elided in the attempted statement of the objectivity theorem above: the operations that we're performing depend on the contribution relation of the latent string model z . Let's call that relation $\blacktriangleright$, as distinct from the relation $\triangleright$ of y . Taken naively, this will make each of the error terms at least as big as $K(\triangleright)$. In some regime—roughly, when the Kolmogorov complexities of some relevant strings is much larger than the Kolmogorov complexity of the contribution relation—we might regard this as not mattering, so the error terms can be seen as negligible. But we'd like to also have something to say when the complexity of the contribution relations can't be neglected.

The simplest step would be to relativize everything to the relation $\blacktriangleright$. That is, we would have something like

$$Kc(y_{\supseteq A} \mid z_{\mathcal{F}^\star}, \blacktriangleright) \leq \sum_{B \in \mathcal{F}} Kc(y_{\supseteq A} \mid x_B, \blacktriangleright) + \sum_{v \in N} I(z_{\mathcal{L}(v)}; z_{\mathcal{R}(v)} \mid z_{\mathcal{I}(v)}, \blacktriangleright) + \text{error terms},$$

which we could combine with

$$\begin{aligned} Kc(y_{\supseteq A} \mid z_{\mathcal{F}^\star}) &\leq Kc(y_{\supseteq A}, \blacktriangleright \mid z_{\mathcal{F}^\star}) + O(1) \\ &\leq Kc(y_{\supseteq A} \mid z_{\mathcal{F}^\star}, \blacktriangleright) + K(\blacktriangleright) + O(1). \end{aligned}$$

Then, only this $K(\blacktriangleright)$ term depends on $\blacktriangleright$, rather than every error term as before.

We should be able to do even better. One way to think of the above situation is that we’re effectively combining the contribution relation $\blacktriangleright$ into the top latent $z_I = z_\star$. Instead, we can distribute the information embodied by the contribution relation among the various latents. We can implement this computationally as follows. We’ve already said that the top latents $y_\star$ and $z_\star$ have to perform the computational roles in their respective models of calculating the given strings x_i from the respective latent strings y_j or z_k . Now, we can further ask them to calculate the contribution relation. Most simply, we can ask that $y_\star(j, i)$ answer the question of whether $j \triangleright i$. By putting the information of the contribution relation into the top latent, this isn’t really an improvement over the previous trick. However, now we can generalize this. We can allow $y_\star(j, i)$ to also answer that it has not yet decided whether $j \triangleright i$, and needs more information—information contained in other latents like some y_ℓ with $\ell \in J$. Then, maybe we ask again by computing $y_\star(j, i, y_\ell)$. There’s a lot of choices to make and bookkeeping to keep track of here. For example, we need some well-foundedness. This should be something like $y_\star$ having already decided previously that $\ell \triangleright i$, in order to prevent the information in y_ℓ from leaking into x_i when it shouldn’t. We need to prevent circularity here. But I hope that by such methods we can prevent the otherwise large overhead associated with the contribution relation.

1.1.2. Other complexity notions. So far, we’ve discussed Shannon information and variants of Kolmogorov complexity (plain complexity, prefix complexity, the modified conditional prefix complexity Kc). For settings where an agent makes observations which can be encoded as strings, we’d like to use complexity notions appropriate to strings, broadly similar to Kolmogorov complexity. However, these Kolmogorov complexities have many more properties than we really use in the proof of the objectivity theorem. Really, all that we need from a complexity measure is that it (approximately) satisfy certain information-theoretic identities and inequalities, things like $H(u, v) = H(u) + H(v \mid u)$, together with certain string bookkeeping identities like $H(u, v) = H(v, u)$.

There are many different potential ways one could approach defining a kind of complexity for strings. I’ll list some that I’m aware of, though others may be attractive for different reasons.

One idea is to use the length of the shortest *straight-line program* as the complexity metric, rather than allowing general computation. There is a lot of work on this that I’m not familiar with, and it has connections to ideas like the widely used Lempel-Ziv compression algorithm, but what I know of it comes from Vanessa Kosoy (Kosoy, 2026). First, in order for straight-line complexity to give an interesting form of condensation, we want to know whether all the necessary information-theoretic properties hold, and with what sort of errors. Next, we can ask about the computational complexity of various operations, most notably finding latent string models with properties like almost-perfect condensation, in cases where they exist. Here, in the case of compressing a single string, there are positive results on efficient computation of close-to-optimal compressions (Rytter, 2003), though there are also negative results (Charikar et al., 2005). This gives some hope that there could be also be efficient computations of interesting latent string models. In any case, optimization over the space of straight-line programs is computable, unlike optimization over all programs for a universal Turing machine.

If we just want a computable variant of algorithmic-information condensation, we can substitute some kind of Kolmogorov-complexity-like measure that also accounts for computation resources, such as by adding a penalty for time or space usage. I expect that there is some version of this such that appropriate algorithmic-information properties hold, allowing us to prove an objectivity theorem. M. Li and Vitányi (2019) have some discussion of Kolmogorov complexity with computational constraints, but I’m not sure if the complexity metrics they define are what we’d need.

Another possibility is to think of complexity using neural nets. Here, the idea is that the complexity of a string would be something like some measure of the complexity of a neural net, plus the surprise of that neural net when asked to predict the string. We can think of the neural nets as trained with local search methods, rather than globally optimal. In such a setting, it’s probably too much to ask for the approximate information-theoretic identities to always hold, but we can investigate whether they usually hold for interesting data. For conditional complexity, the analogue of $K(y | x)$ should be something like the complexity of a network that is allowed to look at (i.e. has connections in from) a frozen copy of a network trained to predict x well. Will $K(x, y) \approx K(x) + K(y | x)$ hold approximately when x and y are two subsets of a pretraining corpus obtained by filtering for different kinds of text, or are input-output pairs of simple algorithmic tasks?

It might also be helpful to consider the situation abstractly. Can we axiomatize some properties that a complexity measure can satisfy, and derive an abstract objectivity inequality? Such ideas may help guide the creation of interesting complexity measure different from those already considered.

1.2. Online and out-of-distribution generalization. Here, we’ll discuss the application of condensation ideas to get new kinds of control of generalization in learning theory. I’m putting this in this section about agency, and I’m giving it this preamble about understanding the behaviour of agents, but the problems discussed here can also be understood as purely learning-theoretic.

An agent must do many things that they can be confident will work well for them on the basis of their past experience of their world. But the world is not a supervised learning problem, where the challenges one encounters are drawn from the same distribution as those one has dealt with in the past, and where it suffices to do well on average. Agents encounter situations that are meaningfully new, where the right courses of action are underdetermined by their experience. When something new happens, learning theory gives us poor control over whether things that worked in the past continue to work in the future. This shows up in the usefulness of novelty-avoiding methods for avoiding catastrophes, such as in Ehtekar and Cohen (2026). In principal, we’d like to do even more; instead of just avoiding novelty, we’d like to know what kinds of novel actions really are desirable for underlying reasons that we can see as shared with past events, and when we’d like to be more cautious, perhaps by offering human correction, of novel actions. What we want here is to understand the hypotheses and behaviour of agents, rather than treating them as black boxes. The realm of reason, argument, and explanation reemerges; in a certain artificial long run, the best predictors win out by virtue of their track record, and their reasoning can fade into the black boxes of their predictive methods, but in the real world, faced with conflicting plausible advice, one wants more.

We can point out what features of existing learning theory make it insufficient for an understanding of these issues. There are two main kinds of learning theory. First, there is the theory of generalization bounds based on data drawn i.i.d. from a fixed distribution. Here, we can get useful guarantees, but as soon as the distribution changes, we are without adequate guidance from theory. Second, there is online learning theory. Here, we have theorems like various sublinear regret bounds. These control the total number of mistakes, but they don't tell us about the particular novel situations where mistakes are more likely. (We can also consider various kinds of reinforcement learning and active learning; for our purposes here, these have the same issues as online learning, but with further complications.) A few ideas give success going beyond either of these strands, such as the work of Aaronson, Lee, and J. Li (2026), but these are isolated hopeful ideas that have not yet given rise to a powerful general theory.

Consider instead the following setting. We have some initial segment $(x_t)_{t=1}^{T_1}$ of data, which is a prefix of a longer segment $(x_t)_{t=1}^{T_2}$ for some $T_2 > T_1$. Suppose that both string models admit latent string models $((y_j)_{j \in J}, \triangleright)$ and $((z_k)_{k \in K}, \blacktriangleright)$ with strong condensation properties. We'll give an example of what it might mean for $(y_j)_{j \in J}$ to have strong condensation properties. First, for $j, k \in J$, we define $j \succeq k$ to mean that for all $t \in [1, T_1]$, if $k \triangleright t$, then also $j \triangleright t$, and also define the shorthand $\{j \triangleright\}$ for the set $\{i : j \triangleright i\}$. Now, we can define the following.

Definition 1. $(y_j)_{j \in J}$ is an (m, r, ε) -almost perfect condensation if (1) $|J| \leq m$, (2) for all $j \in J$ and all $A \subseteq I$, if $|\{j \triangleright\} \cap A| \geq r$, then $Kc(y_{\succeq j} \mid x_A) \leq \varepsilon$, (3) for all $\succeq$ -upward closed sets $\mathcal{F}, \mathcal{G} \subseteq J$, we have $I(y_{\mathcal{F}}; y_{\mathcal{G}} \mid y_{\mathcal{F} \cap \mathcal{G}}) \leq \varepsilon$. Further, it is *well-separated* if (4) for all $j, k \in J$, if $|j \triangleright| \geq 2r$ then at least one of the sets $\{j \triangleright\} - \{k \triangleright\}$ and $\{k \triangleright\} - \{j \triangleright\}$ has cardinality at least $2r$.

So, we might assume for example that both y and z are well-separated (m, r, ε) -almost perfect condensations, for some sufficiently small m, r , and ε . Note that such condensations need not exist, and even if such a y exists, that doesn't guarantee the existence of z , which must account for the longer segment $(x_t)_{t=1}^{T_2}$.

Under such hypotheses, we'd like to be able to prove something about how a universal learner generalizes from $x_{1:T_1}$ to $x_{(T_1+1):T_2}$, even if this constitutes a distributional shift, or more generally if the sequence $x_{1:T_2}$ is far from i.i.d. The idea here can be stated in terms of the relation between y and z . By the objectivity theorem, we should get some sort of correspondence between the latent strings of y and z , with the exceptions of those latents that contribute to few elements of x , which we can neglect as "noise" rather than "pattern", and of z latents which contribute only to observations with indices in the range $[T_1 + 1, T_2]$, which may have no y analogue. Now, we can hope that at those $t \in [T_1 + 1, T_2]$ with no such new "pattern" latents, we can be confident that we have generalized from experience to the best of our abilities. To attempt a more precise statement, we'd like to say that a universal learner might not be confident in a single prediction about what comes next, but their uncertainty should be mostly "noise", in the sense that this question should have low mutual information with other questions that the learner tries to predict, i.e. with other x_t for $t \in [T_1 + 1, T_2]$. Or in another formulation, all algorithmically simple universal learners ought assign similar probability distributions to x_t .

I hope that such a theory could go some way toward an explanation of why learning algorithms in fact do often generalize well out of distribution. The fact that something like this is true is necessary for the success of human learning, and even more clearly for the success of the methods underlying current AI. Even though we don't care whether chatbots are good at predicting the next token, the unsupervised pretraining objective is extremely useful as a component in creating chatbots that answer questions that we do care about. While the fact that it works is so obvious that it's often unacknowledged, this is not promised to us by statistical learning theory!

Such ideas would also provide a starting point for elaboration in various directions. We could get some sort of attribution of knowledge: which previous observations are actually the reason that the learner knows what it knows now? Especially when the underlying things learned are more like values and less like facts, we want to know when what comes next is underdetermined by previous experience.

1.3. Value learning in toy settings. I think it would be informative to see how much could be done with such methods in a value-learning setting like the diamond maximizer problem (Yudkowsky, 2015). It is hard to define the amount of diamond in an environment directly from an agent's observations. Instead, one often wants to learn what diamond is from some sort of corpus of examples. Condensation suggests two directions here. On the one hand, as discussed in the section on generalization and learning theory, we can hope to better understand to what extent concepts are really pinned down by given data, and in what realm they become underdetermined and more specification is needed. On the other hand, if an agent's understanding can be seen as a latent string model, then we can hope to speak the agent's language and describe diamond directly. Perhaps we want to use both these methods together, somewhat like what we do when explaining new ideas to each other.

In particular, we want to be able to tell a diamond maximizer a certain amount about what diamonds are, and then have them learn more about the world, while still pursuing diamond. At least for sufficiently tame kinds of ontological revision, we might hope that the broader structure of certain latent strings survives even as our picture of physics and thus of the microstructure of diamonds changes.

2. EXISTENCE OF CONDENSATIONS

In the above discussion, we focussed on those cases where latent string models with strong condensation properties exist. For the theory of condensation to be useful, we'd like condensation properties that are general enough to apply to interesting data, but narrow enough to imply objectivity-like conclusions. To know that such properties apply to interesting data, we'd like to come up with interesting string models or random variable models satisfying them.

2.1. Almost perfect condensation. One strong condensation property is almost perfect condensation. I've defined this somewhat differently in different places, since I haven't settled on the best way to understand this, and as such I haven't been especially careful about the version that I'm using here, but this should be something like definition 1. Here, we should have an objectivity theorem as follows. If $\mathcal{Y}$ and $\mathcal{Z}$ are two well-separated almost perfect condensations of the same string model, then this should induce a bijection between the latents of $\mathcal{Y}$ and $\mathcal{Z}$ with

good objectivity-style mutual determination. More generally, if $\mathcal{Y}$ and $\mathcal{Z}$ are both almost perfect condensations, and $\mathcal{Y}$ is well-separated, then we get a partition of the latents of $\mathcal{Z}$ into some sets, and these sets admit a bijection with the latents of $\mathcal{Y}$ with similar properties to the previous case. Finally, if we only assume that $\mathcal{Y}$ and $\mathcal{Z}$ are almost perfect condensations, without any well-separatedness assumption, we can still conclude that each latent of $\mathcal{Y}$ is determined by some small set of latents of $\mathcal{Z}$, and vice versa.

I haven't written up the above, partly because my thoughts on exactly which definitions are most natural are still in flux, but I have notes that I can share.

We can construct toy examples of almost-perfect condensations. For example, in the random-variable setting, we can consider the following example, where the latents are the biases of dice, and the given variables are observations that give us some information about these dice. That is, we will have a set of latents $(Y_j)_{j \in \tilde{J}}$, each of which is valued in the space of probability measures on $\{1, \dots, 6\}$, and which are jointly independent. The idea here is that the bias of each die is itself random (we can think of drawing it from a bag of biased dice), but that particular die, with that bias, is rolled multiple times, giving us information about the bias. Now, for the construction, fix some sets $I, \tilde{J}$, and a relation $\triangleright \subseteq \tilde{J} \times I$. Construct a joint distribution on $(Y_j)_{j \in \tilde{J}}$ as described. Define $(X_i)_{i \in I}$ by making a roll $R_{j,i}$ of die j , and setting

$$X_i = \sum_{j \triangleright i} R_{j,i}.$$

We need further latents to account for the idiosyncratic information in X_i , so set $J = \tilde{J} \sqcup I$ and $Y_i = X_i$, extending $\triangleright$ to $J \times I$ by setting also $i \triangleright i$. We can check that for appropriate parameters, $((Y_j)_{j \in J}, \triangleright)$ is an almost perfect condensation of $(X_i)_{i \in I}$. We want $|I|$ sufficiently large relative to $|\tilde{J}|$; not too many $j \in \tilde{J}$ such that $j \triangleright i$ for each fixed $i \in I$ (e.g., we could choose $\triangleright$ such that there are exactly two such j for each i , that is, each observation is the sum of two die rolls); and $\triangleright$ sufficiently evenly distributed, avoiding properties like two different $j \in \tilde{J}$ contributing to the same, or nearly the same, set of $i \in I$.

It would be helpful to have examples of almost perfect condensation that are a bit less artificial, and better demonstrate its relevance to constructing interesting latents. This could be in either the random-variable or the algorithmic-information setting. It would also be good if there were better ways to think about the rather long definition given, in which many somewhat arbitrary choices were made. For example, in definition 1, many things that we want to be small are individually bounded by the same ε . Any bounds would give some kind of objectivity theorem. Is there a better way of thinking about adapting these upper bounds to the different quantities bounded? This might be a situation where the bounds should be adapted to particular latent variable models being studied. But even then, there could be some general insight to be learned from looking at interesting such examples.

2.2. Other kinds of condensation. visual scene, pseudometric, ising model, geometry, few-to-one

3. CONDENSATION AND STATISTICAL MODELS

In statistics, many different kinds of models with latent variables have been studied. I'm interested in the extent to which condensation can serve as a sort of framework here; when latent variable models¹ are useful for understanding data, can we say that part of the reason that they have good condensation properties? Here, I have in mind models such as Bayesian networks with latent variables, sparse autoencoders, hidden Markov models, and various parametric latent variable models (e.g. mixtures of Gaussians).

Often, we care about *identifiability* of parameters of latent variable models. When parameters are underdetermined by data, it is hard to take them seriously as telling us something meaningful about the world—other parameter values would do just as well. The objectivity theorem gives us both less and more than identifiability.

Strictly speaking, it's in a different setting, because identifiability involves taking a limit as data increases. In random-variable condensation, we use a joint distribution, which we could imagine as being estimated from an increasing sample, but a better analogue, especially in the case of algorithmic-information condensation, is to let the *number* of given strings or given random variables increase. Some details depend on exactly what question we ask, but this will very rarely give us a unique element in the whole space of latent string models or latent variable models. However, we also can get more than identifiability, in that *all*² the latent models satisfying certain properties (e.g. almost perfect condensation) will stand in a close relation to each other, not just some parametric or otherwise characterized family. In this way, we can have a confidence in our model without the same dependence on our selection of model class. Though, there are still choices made here, in choosing to be interested in condensation; in choosing a complexity measure; in estimating whether we have found an almost perfect condensation, which is uncomputable if we use genuine Kolmogorov complexity, etc.

While we have been talking about models with latent variables, we should also take note that these ideas can be relevant even to statistical models without latent variables. Given a random variable model $(X_i)_{i \in I}$, we can consider latent variable models where the same family $(X_i)_{i \in I}$ is also the family of latent variable, with the same index set I . This is not trivial, because we still have to choose the contribution relation $\triangleright \subseteq I \times I$. There is a real question, for example, of whether there is some choice of $\triangleright$ such that the resulting latent variable model is an almost perfect condensation. In this way, there is a potential to say something about models like Bayesian networks, fully observed Markov models, and maybe some other graphical models, such as those that appear in statistical physics.

with is Bayesian networks, SAEs, Markov models, factor analysis
ICA? graphical models?

¹*Latent variable models* is here not meant in the specialized sense of Eisenstat (2025), though the senses are close enough that it isn't really reason for confusion.

²Here we do mean latent variable model in our technical sense. There's a methodological point here, visible in this word "all"; we must give the term a technical sense if we want to say something about all latent variable models. This is analogous to how e.g. Turing had to give "algorithm" a definition in order to say that *all* algorithms fail to solve the halting problem.

4. LANGUAGE AND LOGIC

5. FURTHER DIRECTIONS

causation & decision; identifying agents; organization of math (lemmas, structures, basic relations, higher consistency/truth/LC axioms); real string data, music; reflective cooperation, Schelling points; conceptual revision; logic, language

REFERENCES

- Aaronson, Scott, Lin Lin Lee, and Jiawei Li (May 12, 2026). *Sparsity and Out-of-Distribution Generalization*. arXiv: 2603.07388[cs.LG].
- Charikar, M. et al. (July 2005). “The smallest grammar problem.” In: *IEEE Transactions on Information Theory* 51.7, pp. 2554–2576.
- Ebtekar, Aram and Michael K. Cohen (Apr. 15, 2026). *Golden Handcuffs make safer AI agents*. arXiv: 2604.13609[cs.LG].
- Eisenstat, Sam (2025). “Condensation: a theory of concepts.” In: *ODYSSEY 2025 Conference*.
- Kosoy, Vanessa (Mar. 27, 2026). *Stringological sequence prediction I: efficient algorithms for predicting highly repetitive sequences*. arXiv: 2603.26852[cs.FL].
- Li, Ming and Paul Vitányi (2019). *An Introduction to Kolmogorov Complexity and Its Applications*. Texts in Computer Science. Cham: Springer International Publishing.
- Rytter, Wojciech (June 13, 2003). “Application of Lempel–Ziv factorization to the approximation of grammar-based compression.” In: *Theoretical Computer Science* 302.1, pp. 211–222.
- Yudkowsky, Eliezer (Dec. 18, 2015). *Diamond maximizer — Less Wrong*. URL: <https://www.lesswrong.com/w/diamond-maximizer>.